Relatório 04 - Prática: Principais Bibliotecas e Ferramentas Python para Aprendizado de Máquina (I)

João Victor Silveira Vasiliauskas

1. Introdução

O objetivo deste relatório é explicar sobre o conteúdo do curso Python para Data Science e Machine Learning, especificamente as seções 3, 5, 6 e 7. Na qual envolvem a análise de dados com uso das bibliotecas do Python: Numpy e Pandas. Usando a IDE Jupyter Notebook.

2. Desenvolvimento

Para começar, devo explicar as diferenças entre NumPy e Pandas:

Diferenças entre:	NumPy	Pandas
Utilização	Algebra Linear Numérica	Manipulação, tratamento e análise de Dados
Estrutura de Dados	ndarrays(arrays multidimensionais)	Series e DataFrames
Tipos de Dados	Homogêneo(mesmo tipo de dado percorre a estrutura)	Heterogêneo(diferentes tipos de dado percorre a estrutura)
Rótulos	Índices Númericos	Rótulos explícitos, personalizáveis
Dependência	Biblioteca independente.	Biblioteca construída em cima do NumPy

Imagem 1 - Diferenças entre NumPy e Pandas, Autoria Própria

OBS: As bibliotecas escolhidas são usadas por serem de código aberto e terem suas funcionalidades compiladas em C.
Além disso, devo explicar a IDE usada:
Jupyter Notebook: Jupyter Notebook é uma plataforma web de código aberto feita para auxiliar computação científica e análise de dados. Sua principal vantagem é a capacidade de rodar linhas de código iterativamente, ou seja, ao invés de rodar todo o código, ele pode rodar uma linha por vez.

Com isso em mente, podemos começar o detalhamento de cada biblioteca e suas capacidades.

NumPy: Dentro da biblioteca NumPy temos a estrutura de dados central e algumas funções essenciais para seu uso, essa estrutura seria:

ndarray: um array multidimensional utilizado para análises de um conjunto de dados de mesmo tipo(homogêneo).

Além disso, as funções essenciais dentro do NumPy são:

Criação de arrays:
- array(): cria um array com uma tupla ou uma lista;
- arange(x, y, z): cria um array que recebe x como partida, y como parada e z como o número a incrementar da partida a parada;
- zeros(): cria um array onde todo elemento é zero;
- ones(): cria um array onde todo elemento é um;
- eye(): cria um array bidimensional em formato identidade(diagonal principal com 1s e o resto 0);
- linspace(x, y, z): cria um array parecido com arange, onde x é a partida e y é a parada, só que o z é o número de valores entre x e y, com todos os valores sendo equidistantes do seu antecessor e seu sucessor;
- Criação de dados randomizados:
    - random.rand(): cria um array multidimensional ou não, com valores aleatórios.
    - random.randn(): cria um array multidimensional ou não, com valores de média 0 e variância 1.
    - random.randint(): cria um array multidimensional ou não, com valores entre o mínimo e o máximo selecionados dentro do método.

Manipulação de arrays:
- shape(): mostra qual o tamanho do array;
- round(): arredonda os números dentro do array até x casas decimais
- reshape(x, y): permite que mudar um array bidimensional de tamanho (w, z) para um array de tamanho diferente, x; y, desde que w*z seja igual a x*y;
- max(): mostra o maior valor dentro de um array;
- min(): mostra o menor valor dentro de um array;

- argmax(): mostra o índice do maior valor dentro de um array;
- argmin(): mostra o índice do menor valor dentro de um array;
- copy(): permite copiar os valores do array, útil para quando você precisa criar duas instâncias separadas do array.(Caso um array ou parte de um array seja colocado em um outro array sem o uso de copy(), quaisquer alterações nesse array afetam o array principal);

Indexação de arrays:

- Unidimensional:
  - array[x]: revela o elemento de índice x
- Bidimensional:
  - array[x][y] ou array[x, y]: revela o elemento na linha x, coluna y;
  - array[condição]: revela apenas os valores que cumprem a condição;
- OBS: Funciona da mesma forma que indexação de arrays de Python, isso é, é possível percorrer um array usando array[:] e ao colocar valores na frente ou atrás dos dois pontos, define os limites;

Operações com arrays:

- Aritméticas:
  - Array e Valor: aplica às operações básicas(soma, subtração, multiplicação, divisão e exponenciação) a cada um dos valores do array com o valor definido;
  - Array e Array: aplica às operações básicas a cada um dos valores do array a partir do índice, aplicando a operação no índice n do array1 com o índice n do array2, por isso, os arrays devem ser de mesmo tamanho total;
  - Função dentro do Array: contém as seguintes funções: sqrt(raiz quadrada), exp(exponencial), mean(média), std(desvio padrão populacional), sin(seno);
  - OBS1: Divisões por 0 resulta em aviso e com o número onde a divisão ocorreu igual a nan(nulo);
- Booleanas:
  - and: Operações booleanas que normalmente usariam o “and” para indexar usam o símbolo “&”;
  - or: Operações booleanas que normalmente usariam o “or” para indexar usam o símbolo “|”;

Assim terminamos o essencial da biblioteca numpy, seguindo em frente, temos a biblioteca pandas.

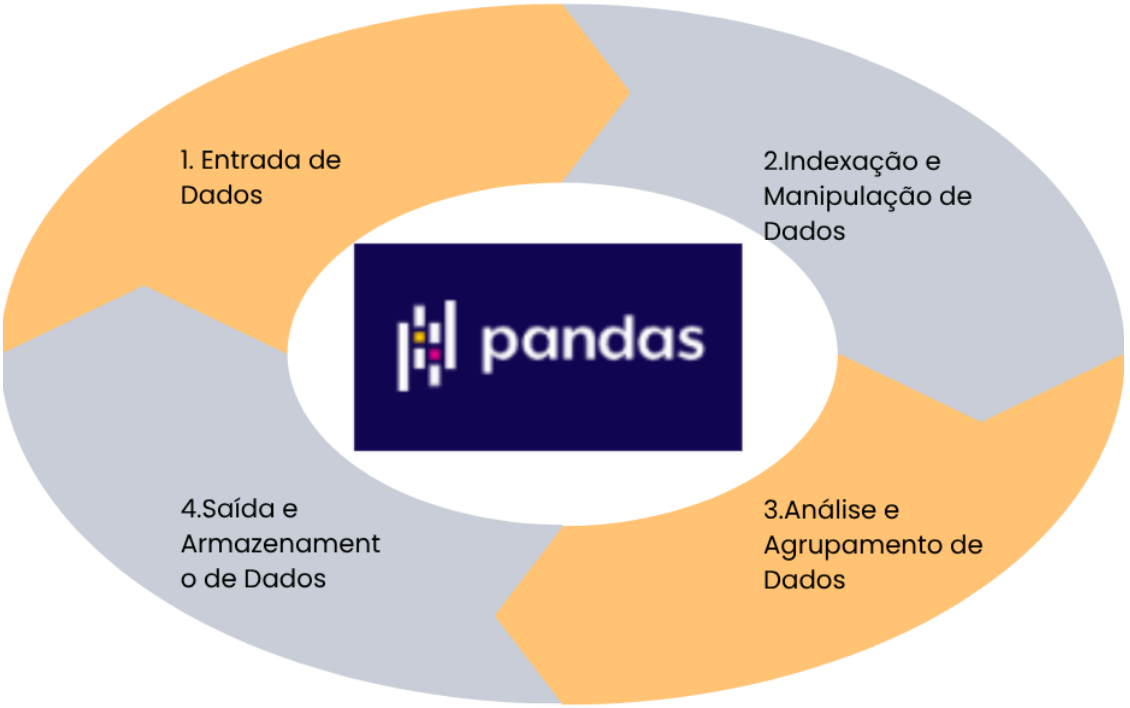


Imagem 2 - Ciclo de Dados na biblioteca Pandas.
Fonte: Adaptado de Equal Data (2026).

1. **Entrada de Dados:**
    - Apenas Series:
        - Series(data, index): cria um array unidimensional com índices, tanto para data quanto para index ele recebe arrays unidimensionais(listas e tuplas) ou um dict;
    - Apenas DataFrames:
        - DataFrame(data, index, columns): cria um DataFrame recebendo listas, tuplas e dicionários como data, índices e colunas(apenas listas e tuplas para índices e colunas);
        - read_<tipo_arquivo>(): cria um DataFrame que recebe seus valores através de um certo tipo de arquivo, os principais são:
            - read_csv(): cria um DataFrame que recebe seus valores através de um arquivo csv (caso os dados não estejam separados por vírgula, o parâmetro sep deve ser igual ao valor específico);
            - read_excel(): cria um DataFrame que recebe seus valores através de um arquivo excel (se houver mais de uma planilha, ela deve ser definida dentro da função);
            - read_html(): cria um DataFrame que recebe seus valores através de um arquivo html ou links de website;

2. **Indexação e Manipulação de Dados:**
    - Apenas Series ou DataFrames indexado em apenas uma coluna:
        - unique(): mostra os números distintos presentes em uma Series inteira(mostra apenas um dos eixos do DataFrame);
        - nunique(): mostra o número de valores distintos presentes dentro de uma Series inteira(linha/coluna do DataFrame);
        - value_counts(): mostra os números de valores distintos e cada um dos valores distintos dentro da Series(linha/coluna de um DataFrame);
        - Transformar duas Series/DataFrames em um:
            - concat(): pega um eixo para realizar a concatenação, e combina todas as Series/DataFrames;
    - Apenas DataFrames:
        - [“C”]: permite ver a coluna C;
        - [[“C0”, “C1”]]: permite ver as colunas dentro dos valores colocados;
        - .C: pode ser usado para ver a coluna selecionada(nesse caso, C), não recomendado já que pode ser confundida com um método;
        - set_index(Transforma a coluna marcada no índice)(Precisa do Inplace para a alteração ocorrer no DataFrame);
        - [“NOVACOLUNA”] = dados: Cria uma nova coluna com o nome colocado dentro das barras “[]” com os dados colocados;
        - Transformar dois DataFrames em um:
            - merge(): mescla todos os DataFrames desde que contenham um elemento em comum;
            - join(): junta todos os DataFrames que podem não conter um elemento em comum, usa o objeto como referência para juntar o DataFrame inserido;
            - OBS: o merge() e join() podem usar o atributo “how”, que permite a alteração do método de combinação:
            - inner: Usa a intersecção de chaves de ambos os frames, baseado em junções relacionais sobre chaves/rótulos coincidentes;
            - outer: Usa a união de chaves de ambas os frames, baseado em junções relacionais sobre chaves/rótulos coincidentes;

        - left: Usa a união de chaves do DataFrame a esquerda;
        - right: Usa a união de chaves do DataFrame a direita;
- Ambos:
    - loc(): permite localizar os valores dentro dos labels definidos;
    - iloc(): permite usar índices ao invés de usar labels;
    - Tratamento de dados ausentes:
        - dropna: retira todas as linhas e colunas que contêm um valor com NaN, recomendado para estritamente remover dados;
        - fillna: preenche os dados vazios com um dado de escolha;
        - ffill(forward fill): método que preenche os dados nan com dados que o precedem;
        - bfill(backward fill): método que preenche os dados nan com dados que o sucedem;
        - OBS: Todos esses métodos têm que ter inplace True para afetarem o DataFrame;
    - apply(): aplica uma função dentro de cada um dos valores dentro de uma linha ou coluna(no caso de um DataFrame), e aplica em todos os valores dentro de uma Séries;
- drop(labels, axis, inplace): Retira os rótulos colocados no labels, verificando em qual eixo(axis) e afeta a Series ou DataFrame apenas se for usado com o inplace = True;
- reset_index: Reseta os índices, no qual precisa do Inplace para a alteração ocorrer no Series ou DataFrame;
- Índices Multiníveis:
    - MultiIndex.from_tuples(): converte uma lista de tuplas em índices multiniveis;
    - xs: faz uma análise de seção transversal, usado para acessar os índices mais externos dentro de multiníveis;

3. **Análise e Agrupamento de Dados**
    - Ambos:
        - groupby(): Usa o índice colocado dentro do método para fazer operações com o grupo de dados;
        - Métodos para análise de dados:
            - mean: calcula a média amostral do DataFrame;
            - std: calcula o desvio padrão amostral da estrutura;
            - max: calcula o valor máximo amostral da estrutura;
            - min: calcula o valor mínimo amostral da estrutura;

        - sum: calcula a soma de cada amostra da estrutura;
        - describe: gera estatísticas como média, desvio padrão, quartis, mín e máx de uma vez;
    - Operações Booleana:
        - Estrutura[estrutura > 0];
        - Ao invés de usar os comparadores usando seus nomes: and(e),or(ou), usamos os seus símbolos: &, |;
- Apenas em DataFrame:
        - DataFrame[DataFrame[“C0”] > 0];
        - DataFrame[DataFrame[“C0”] > 0][“C2”];

4. **Saída e Armazenamento de Dados**
    - Ambos:
        - to_<tipo_arquivo>(): Exporta a estrutura de dados(Series e DataFrame) para o tipo de arquivo específico, os principais são: to_csv , to_excel, to_html;

3. Conclusão

Com isso, podemos concluir que mesmo o Pandas sendo a biblioteca mais utilizada para análises de dados em geral, ela não seria capaz de existir sem a biblioteca NumPy, onde ambos podem seguem o ciclo de vida dos dados, indo da entrada de dados para a sua saída e .

4. Referências

EQUAL DATA. *Entenda o ciclo de vida dos dados*. Blog Equal Data. Disponível em: https://equaldata.ai/blog/entenda-o-ciclo-de-vida-dos-dados/. Acesso em: 17 set. 2026.

NEVES, Thiago. **Python para Data Science e Machine Learning completo.** Udemy, [s.d.]. 1 curso online. Acesso restrito a alunos matriculados.

NUMPY DEVELOPERS. *NumPy: The fundamental package for scientific computing with Python*. Disponível em: https://numpy.org/. Acesso em: 14 set. 2026.

PANDAS DEVELOPMENT TEAM. *pandas: Python Data Analysis Library*. Disponível em: https://pandas.pydata.org/. Acesso em: 14 set. 2026.

PYTHON SOFTWARE FOUNDATION. *Python Language Reference*. Disponível em: https://www.python.org/. Acesso em: 14 set. 2026.